

RETAILMART V3 • SQL

SQL JOINS Part 2 — FULL OUTER + Multi-Table

WhatsApp Announcement

JOINS PART 2 — FULL OUTER + MULTI-TABLE

Yesterday: INNER, LEFT, RIGHT. Today: FULL OUTER (the 'reconciliation' JOIN) and multi-table JOINS that span 4-5 tables. This is how Amazon's order page works — products + customers + stores + warehouses + payments in ONE query. Today you start writing queries that real analysts ship to production!

Today's Focus:

- FULL OUTER JOIN — keep everything from both tables (reconciliation)
- Multi-table JOINS — chaining 4, 5+ tables in one query
- JOIN order intuition — pick the anchor, chain outward
- Rewriting JOINS three different ways (interview classic)

After today, you can answer ANY 'find data across our entire system' question.

— Siraj Sir

#Day10 #PostgreSQL #FullOuterJoin #MultiTableJoins

TODAY'S AGENDA (2 Hours)

Section	Time	Topics
Quick Recap	5 mins	INNER / LEFT / RIGHT recap, Anti-join pattern review
Part 1: FULL OUTER JOIN	45 mins	FULL OUTER JOIN — keep ALL rows from BOTH sides, Reconciliation use cases (orders vs payments), Match classification with CASE, 4 practice problems (Easy → Crazy)
Part 2: Multi-Table JOINS (4-5 tables)	50 mins	Chaining JOINS along FK paths, Star pattern vs chain pattern, Supply-chain trace: orders → shipments → warehouses → inventory, 4 practice problems (Easy → Crazy)
Part 3: JOIN Strategy & Rewriting	25 mins	JOIN decision flowchart, Rewriting a 3-table JOIN three different ways, Reading EXPLAIN to choose the right approach
Quiz + Summary & Wrap-up	15 mins	Try It Yourself challenges, Quick quiz, common mistakes, interview bank, Homework + Subqueries Part 1 preview

YOUR SQL JOURNEY



Filtering & Sorting



Scalar Functions



CASE & COALESCE



Aggregates & GROUP BY



JOINS Part 1



JOINS Part 2

← HERE



Self Join & UNION



Subqueries Part 1

Memory Check

INNER JOIN (only matches), LEFT JOIN (all from left + NULLs), RIGHT JOIN (mirror), and 3-table multi JOINS. Anti-join pattern (LEFT + WHERE IS NULL) for finding missing data.

Today: FULL OUTER JOIN completes the JOIN family — preserve unmatched rows from BOTH sides. Then multi-table JOINS spanning 4-5 tables in one query. The skill that turns SQL learners into production analysts.

The Reconciliation Problem

Your bank says you have ₹50,000 in your account. Your accountant's records show ₹48,000. Which transactions are MISSING from each side? You can't use INNER JOIN (only matches), can't use LEFT (only your view), can't use RIGHT (only their view). You need EVERYTHING — matched AND unmatched from both sides. That is FULL OUTER JOIN.

Then imagine: 'Show me everything about Order #1234' — customer name, product details, store location, payment status, shipment status, warehouse, inventory level. That is FIVE+ tables stitched together. Today, both.

From Story to SQL!

Bank reconciliation — FULL OUTER JOIN bank vs ledger

Find missing transactions — WHERE one side IS NULL

Full order details — JOIN orders + customers + items + products + stores

Supply chain trace — JOIN warehouses + inventory + products + shipments

All FK relationships followed in one query

The Voter ID Audit

Election commission has TWO lists: registered voters AND people who actually voted. FULL OUTER JOIN shows all three categories: registered AND voted (matched), registered but didn't vote (left-only), voted but not registered (right-only — potential fraud). Nothing excluded — exactly what an auditor needs.

FULL OUTER JOIN

Technical:

FULL OUTER JOIN returns ALL rows from BOTH tables. Matching rows are combined. Left-only rows have NULLs for right columns. Right-only rows have NULLs for left columns. FULL JOIN is shorthand for FULL OUTER JOIN.

Layman's:

FULL OUTER JOIN = 'Show EVERYTHING from BOTH tables. Match where you can. Where you cannot, still show the row with NULLs on the missing side.' Used for audits, reconciliations, and 'find the gaps' analyses.

SYNTAX: FULL OUTER JOIN (1/2)

```
-- Basic FULL OUTER JOIN
SELECT a.col1, b.col2
FROM table_a a
FULL OUTER JOIN table_b b
    ON a.key = b.key;

-- FULL JOIN (shorthand – identical)
FULL JOIN table_b b ON a.key = b.key;

-- Find ONLY the unmatched rows (the 'gaps')
FROM a FULL JOIN b ON a.key = b.key
WHERE a.key IS NULL OR b.key IS NULL;

-- Classify match status with CASE
SELECT CASE
    WHEN a.key IS NOT NULL AND b.key IS NOT NULL
        THEN 'Matched'
    WHEN a.key IS NOT NULL THEN 'Left Only'
    ELSE 'Right Only'
END AS status
```

SYNTAX: FULL OUTER JOIN (2/2)

```
FROM a FULL JOIN b ON a.key = b.key;
```

Step-by-Step: How FULL OUTER JOIN Works

```
-- orders:                payments:
-- order_id | total  payment_id | order_id | amt
-- -----|-----
-- 1001      | 5000   P1          | 1001      | 5000
-- 1002      | 3000   P2          | 1003      | 2000

-- FULL OUTER JOIN on order_id:
-- 1001 + P1 (5000)    ✓ MATCHED
-- 1002 + NULL         ← LEFT ONLY (unpaid!)
-- NULL + P2 (1003)    ← RIGHT ONLY (orphan!)

-- Result: 3 rows — nothing excluded
```

Easy: Departments vs Employees Audit

EASY

SCENARIO: The HR Director needs a data quality check before the year-end review. Some departments might have zero employees, and some employees might have an invalid dept_id. She needs a full reconciliation showing every department and every employee.

YOUR TASK: FULL OUTER JOIN dim_department with employees on dept_id. Show dept_name, employee name, and classify each row as 'Matched', 'Dept — No Employees', or 'Employee — Invalid Dept'.

 **Hint:** After a FULL OUTER JOIN, either side can have NULLs — you'll want a single 'identifier' column regardless of which side matched. Trap: selecting plain d.dept_id will return NULL for orphan employees; you need a function that picks the first non-NULL.

Answer



ANSWER

```
SELECT
  COALESCE(d.dept_id, e.dept_id)
    AS dept_id,
  d.dept_name,
  e.first_name || ' ' || e.last_name
    AS employee,
  CASE
    WHEN d.dept_id IS NOT NULL
      AND e.employee_id IS NOT NULL
      THEN 'Matched'
    WHEN d.dept_id IS NOT NULL
      THEN 'Dept – No Employees'
    ELSE 'Employee – Invalid Dept'
  END AS status
FROM core.dim_department d
FULL OUTER JOIN stores.employees e
  ON d.dept_id = e.dept_id
ORDER BY status, dept_id
LIMIT 25;
```

Medium: Orders vs Payments Reconciliation

MEDIUM

SCENARIO: Finance is closing the quarter. They need to reconcile orders against payments — find unpaid orders and orphan payments. A summary report showing counts and totals per category.

YOUR TASK: FULL OUTER JOIN orders with payments on order_id. Use CASE to classify as Matched, Unpaid Order, or Orphan Payment. GROUP BY classification for a summary with counts and totals.

 **Hint:** You're grouping by a computed category, not a column — think about what GROUP BY needs when the grouping expression doesn't have a name yet. Trap: PostgreSQL doesn't accept SELECT aliases in GROUP BY — repeat the entire CASE expression there.

Answer (1/2)

✓ ANSWER

```
SELECT
  CASE
    WHEN o.order_id IS NOT NULL
      AND p.payment_id IS NOT NULL
      THEN 'Matched'
    WHEN o.order_id IS NOT NULL
      THEN 'Unpaid Order'
    ELSE 'Orphan Payment'
  END AS recon_status,
  COUNT(*) AS row_count,
  COALESCE(ROUND(
    SUM(o.net_total), 0), 0)
  AS order_value,
  COALESCE(ROUND(
    SUM(p.amount), 0), 0)
  AS payment_value
FROM sales.orders o
FULL OUTER JOIN sales.payments p
  ON o.order_id = p.order_id
GROUP BY
  CASE
    WHEN o.order_id IS NOT NULL
```


Answer (2/2)

 ANSWER

```
        AND p.payment_id IS NOT NULL
        THEN 'Matched'
    WHEN o.order_id IS NOT NULL
        THEN 'Unpaid Order'
    ELSE 'Orphan Payment'
END
ORDER BY row_count DESC;
```

Hard: Orders vs Shipments — Delivery Audit

HARD

SCENARIO: The Head of Logistics suspects data gaps: some orders might have no shipment record, and some shipments might reference orders that do not exist. She needs a full audit with counts per status category.

YOUR TASK: FULL OUTER JOIN orders with shipments on order_id. Classify: Shipped, Awaiting Shipment, Orphan Shipment. Show per-category count and the oldest pending date.

🤔 **Hint:** The 'oldest pending' is just an aggregate over the unmatched rows. Trap: cancelled orders also have no shipment — without filtering them out, your 'awaiting shipment' bucket gets polluted with orders that intentionally have no shipment.

Answer (1/2)

 ANSWER

```
SELECT
CASE
  WHEN o.order_id IS NOT NULL
    AND sh.shipment_id IS NOT NULL
    THEN 'Shipped'
  WHEN o.order_id IS NOT NULL
    AND o.order_status != 'Cancelled'
    THEN 'Awaiting Shipment'
  WHEN o.order_id IS NOT NULL
    THEN 'Cancelled – No Shipment'
  ELSE 'Orphan Shipment'
END AS shipment_status,
COUNT(*) AS row_count,
MIN(o.order_date) AS oldest_date
FROM sales.orders o
FULL OUTER JOIN sales.shipments sh
  ON o.order_id = sh.order_id
GROUP BY
CASE
  WHEN o.order_id IS NOT NULL
    AND sh.shipment_id IS NOT NULL
    THEN 'Shipped'
```

Answer (2/2)

 ANSWER

```
    WHEN o.order_id IS NOT NULL
      AND o.order_status != 'Cancelled'
      THEN 'Awaiting Shipment'
    WHEN o.order_id IS NOT NULL
      THEN 'Cancelled – No Shipment'
    ELSE 'Orphan Shipment'
  END
ORDER BY row_count DESC;
```

Crazy: Payments Reconciliation — sales.payments vs finance.payments

CRAZY

SCENARIO: The CFO is auditing two payment systems: sales.payments (POS-side records) and finance.payments (accounting-side records). Both should agree on every order. Find orders where the two sources disagree — different amounts, missing on one side, or refund mismatches.

YOUR TASK: FULL OUTER JOIN sales.payments to finance.payments on order_id. Classify into Matched / POS-only / Finance-only / Amount-mismatch. Summarize counts and value gaps per category.

🤔 **Hint:** Real-world reconciliation finds FOUR kinds of issues, not just two — both-sides-present-but-different-amount is its own category. Trap: comparing numeric amounts directly can fail on rounding — wrap with ROUND or check $ABS(a - b) < 0.01$ to handle paise-level drift between systems.

Answer (1/2)

✓ ANSWER

```
SELECT
  CASE
    WHEN sp.payment_id IS NULL
      THEN 'Finance Only (missing POS)'
    WHEN fp.payment_id IS NULL
      THEN 'POS Only (missing finance)'
    WHEN ROUND(sp.amount, 2)
      != ROUND(fp.amount, 2)
      THEN 'Amount Mismatch'
    ELSE 'Matched'
  END AS recon_status,
  COUNT(*) AS record_count,
  ROUND(SUM(COALESCE(sp.amount, 0)
    - COALESCE(fp.amount, 0)), 2)
    AS net_gap
FROM sales.payments sp
FULL OUTER JOIN finance.payments fp
  ON sp.order_id = fp.order_id
GROUP BY
  CASE
    WHEN sp.payment_id IS NULL
      THEN 'Finance Only (missing POS)'
```

Answer (2/2)

✓ ANSWER

```
    WHEN fp.payment_id IS NULL
      THEN 'POS Only (missing finance)'
    WHEN ROUND(sp.amount, 2)
      != ROUND(fp.amount, 2)
      THEN 'Amount Mismatch'
    ELSE 'Matched'
  END
ORDER BY record_count DESC;
```

FULL OUTER JOIN

FULL OUTER JOIN returns ALL rows from BOTH tables — nothing excluded. Matched rows combine; unmatched rows get NULLs on the missing side. Primary use: data reconciliation, audits, gap analysis. COALESCE picks the non-NULL key. CASE classifies match status. Used in ~5% of analyst queries — specialized but essential for data quality.

FULL OUTER JOIN in Production

Real-world FULL OUTER JOIN use cases:

- Paytm: Reconcile bank transactions vs internal payment records
- HDFC Bank: Match credit card statements vs merchant payouts
- Razorpay: Find orphan payments (payment without an order)
- Tally/Zoho: Compare expected GST vs filed GST returns

Any time TWO systems must agree on a single source of truth, FULL OUTER JOIN finds the discrepancies.

FULL OUTER JOIN with WHERE on One Side

The Mistake:

FULL OUTER JOIN orders o ... WHERE o.order_status = 'Delivered'. This removes orphan-right rows (where o.order_status IS NULL because there's no matching order) and silently converts FULL into a partial-LEFT-style JOIN.

The Fix:

Side-specific filters belong in the ON clause, OR use COALESCE in WHERE: WHERE COALESCE(o.order_status, 'Unknown') = 'Delivered'. Better still: pre-filter in a subquery, then FULL OUTER JOIN the cleaned sets.

The Complete Order Story

A customer calls support: 'Where is my order?' The agent needs to see EVERYTHING in one screen — customer details, products ordered, payment mode, shipment status, warehouse origin, store branch. That's 6+ tables in a single query, all stitched together by foreign keys.

Multi-table JOINS are how analyst dashboards work. Once you can navigate 4–5 tables comfortably, you can answer almost any business question.

Star Pattern vs Chain Pattern

Star pattern (most common):

One central FACT table (orders) joined to multiple DIMENSION tables (customers, stores, products). Each dimension branches outward like spokes.

Chain pattern:

Following FK relationships step-by-step (employee → store → region). Each table depends on the previous one in the chain.

Real queries usually MIX both. JOIN types can also mix: INNER for required relationships, LEFT for optional ones.

SYNTAX: Multi-Table JOIN Patterns (1/2)

```
-- Star pattern (one anchor, many dimensions)
SELECT o.order_id, c.name, s.store_name,
       p.product_name, oi.quantity
FROM sales.orders o
JOIN customers.customers c
    ON o.cust_id = c.customer_id
JOIN stores.stores s ON o.store_id = s.store_id
JOIN sales.order_items oi
    ON o.order_id = oi.order_id
JOIN products.products p
    ON oi.prod_id = p.product_id;

-- Chain pattern (follow FK path)
SELECT e.first_name, s.store_name, r.region_name
FROM stores.employees e
JOIN stores.stores s ON e.store_id = s.store_id
JOIN core.dim_region r
    ON s.region_id = r.region_id;

-- Mix INNER (required) + LEFT (optional)
```

SYNTAX: Multi-Table JOIN Patterns (2/2)

```
JOIN ... ON ...      -- must match  
LEFT JOIN ... ON ...  -- optional, keep nulls
```

Easy: Full Order Receipt — 4-Table Star

EASY

SCENARIO: Customer Service needs a printable receipt: order_id, customer name, store name, product name, quantity, amount. Four tables in one query — the basic 'order details' dashboard.

YOUR TASK: Anchor on orders. Chain to customers (cust_id), stores (store_id), order_items (order_id), and products (prod_id). Filter to delivered orders. Show 10 most recent.

🤔 **Hint:** Star pattern — orders sits in the middle, each dimension JOINS to it (except products which chains through order_items). Trap: an order has MANY items, so 'one row per order' is impossible here — your output will have multiple rows per order, one per item.

Answer



ANSWER

```
SELECT
  o.order_id,
  o.order_date,
  c.first_name || ' ' || c.last_name
    AS customer,
  s.store_name,
  p.product_name,
  oi.quantity,
  oi.net_amount
FROM sales.orders o
JOIN customers.customers c
  ON o.cust_id = c.customer_id
JOIN stores.stores s
  ON o.store_id = s.store_id
JOIN sales.order_items oi
  ON o.order_id = oi.order_id
JOIN products.products p
  ON oi.prod_id = p.product_id
WHERE o.order_status = 'Delivered'
ORDER BY o.order_date DESC
LIMIT 10;
```


Medium: Unsold Inventory — Anti-Join Across Tables

MEDIUM

SCENARIO: The Inventory Manager wants dead stock: products currently sitting in warehouses but NEVER ordered. This is the anti-join pattern stretched across multiple tables.

YOUR TASK: Start from products. LEFT JOIN to order_items on prod_id. Filter to rows where no order exists. JOIN to dim_brand for the brand name. Show product_name, brand, price.

🤔 **Hint:** 'Products never ordered' is just LEFT JOIN + WHERE IS NULL — extended across one more table for branding. Trap: products has no 'category' column; if you want category, JOIN through dim_brand to dim_category — not directly on products.

Answer

 ANSWER

```
SELECT
    p.product_id,
    p.product_name,
    b.brand_name,
    p.price
FROM products.products p
JOIN core.dim_brand b
    ON p.brand_id = b.brand_id
LEFT JOIN sales.order_items oi
    ON p.product_id = oi.prod_id
WHERE oi.order_item_id IS NULL
ORDER BY p.price DESC
LIMIT 20;
```

Hard: Supply Chain Trace — 5-Table Chain

HARD

SCENARIO: The Head of Supply Chain wants to trace a product from warehouse to customer order: product → supplier shipment → warehouse → current inventory snapshot. Five tables connected by foreign keys — the canonical supply-chain trace query.

YOUR TASK: Start from products. JOIN to supply_chain.shipments (by product_id), then supply_chain.warehouses (by warehouse_id), then supply_chain.inventory_snapshots (latest snapshot). Show product, warehouse, region, shipped quantity, current stock.

🤔 **Hint:** Trace the FK path on paper FIRST — every JOIN must follow a real foreign key, not a same-named column. Trap: inventory_snapshots has a composite primary key (warehouse_id + product_id + snapshot_date) — multiple snapshots per product can fan out unless you filter to a specific date.

Answer (1/2)

✓ ANSWER

```
SELECT
    p.product_name,
    w.name AS warehouse,
    w.location_city,
    w.region,
    sh.quantity AS shipped_qty,
    sh.shipped_date,
    inv.quantity_on_hand
    AS current_stock
FROM products.products p
JOIN supply_chain.shipments sh
    ON p.product_id = sh.product_id
JOIN supply_chain.warehouses w
    ON sh.warehouse_id = w.warehouse_id
LEFT JOIN supply_chain.inventory_snapshots inv
    ON inv.warehouse_id = w.warehouse_id
    AND inv.product_id = p.product_id
    AND inv.snapshot_date = (
        SELECT MAX(snapshot_date)
        FROM supply_chain.inventory_snapshots
    )
WHERE sh.status = 'Delivered'
```

Answer (2/2)

 ANSWER

```
ORDER BY sh.shipped_date DESC  
LIMIT 15;
```

Crazy: 360° Order Report — 7-Table Star with LEFT JOIN

CRAZY

SCENARIO: Customer Care has a complaint: 'Where is my order #X?' The agent needs ONE query that shows: customer info, store branch, product details, payment status, shipment status, and the warehouse the product shipped from. SEVEN tables — the full 360° view.

YOUR TASK: Anchor on orders. INNER JOIN required dimensions (customers, store, items, products). LEFT JOIN optional ones (payments may be pending, shipments may not exist yet, warehouse may be unknown for some products).

🤔 **Hint:** Pick the JOIN type per relationship — required dimensions use INNER, optional ones use LEFT. Trap: a single INNER JOIN to an incomplete table silently drops rows from the entire report — be deliberate about which JOINS are required vs optional.

Answer (1/2)

✓ ANSWER

```
SELECT
  o.order_id,
  o.order_date,
  o.order_status,
  c.first_name || ' ' || c.last_name
    AS customer,
  s.store_name,
  p.product_name,
  oi.quantity,
  oi.net_amount,
  pay.payment_mode,
  pay.amount AS paid_amount,
  sh.courier_name,
  sh.status AS shipment_status,
  w.name AS warehouse_name
FROM sales.orders o
JOIN customers.customers c
  ON o.cust_id = c.customer_id
JOIN stores.stores s
  ON o.store_id = s.store_id
JOIN sales.order_items oi
  ON o.order_id = oi.order_id
```

Answer (2/2)

✓ ANSWER

```
JOIN products.products p
  ON oi.prod_id = p.product_id
LEFT JOIN sales.payments pay
  ON o.order_id = pay.order_id
LEFT JOIN sales.shipments sh
  ON o.order_id = sh.order_id
LEFT JOIN supply_chain.shipments sc_sh
  ON sc_sh.product_id = p.product_id
LEFT JOIN supply_chain.warehouses w
  ON sc_sh.warehouse_id = w.warehouse_id
WHERE o.order_status = 'Delivered'
ORDER BY o.order_date DESC
LIMIT 10;
```


Multi-Table JOINS

Star pattern: one central fact + many dimensions. Chain pattern: $A \rightarrow B \rightarrow C$ following FKs. MIX JOIN types — INNER for required relationships, LEFT for optional. Each JOIN multiplies rows if it's one-to-many. Use pre-aggregation (Subqueries Part 2) when SUMs get inflated. Trace the FK path on paper before writing the SQL.

Multi-Table JOINS Everywhere

Every app you use runs multi-table JOINS constantly:

- Zomato order page: restaurant + menu + items + delivery + payment (5+ tables)
- Amazon order details: products + sellers + warehouses + shipping + payment (6+ tables)
- Bank statement: accounts + transactions + branches + categories (4 tables)
- LinkedIn profile: users + experience + education + skills + connections (5+ tables)

Every screen with 'all the info about X' is a multi-table JOIN.

Row Multiplication (Fan-Out)

The Mistake:

JOIN orders to BOTH order_items AND payments directly: 1 order $\times$ 3 items $\times$ 2 payments = 6 rows per order. COUNT(*) gives 6 instead of 1. SUM doubles/triples real amounts.

The Fix:

Pre-aggregate one side in a subquery: LEFT JOIN (SELECT order_id, COUNT(*) FROM order_items GROUP BY order_id) sub. This gives exactly 1 row per order. We formalize this pattern on Subqueries Part 2.

JOIN Decision Flowchart (1/2)

Q1: Do I need rows from BOTH tables to match? → YES = INNER JOIN

'Show orders with customer names' — both must match.

Q2: Do I need ALL rows from one specific table? → YES = LEFT JOIN

'Show ALL customers, even those with no orders.'

Q3: Do I need unmatched rows from BOTH sides? → YES = FULL OUTER JOIN

'Reconcile orders vs payments — find gaps on both sides.'

JOIN Decision Flowchart (2/2)

Q4: Do I need to find rows that do NOT match? → Use Anti-join
LEFT JOIN + WHERE right.key IS NULL = 'find what is missing'.

Rewriting a 3-Table JOIN — Three Ways (1/2)

```
-- ONE business question, THREE valid SQL forms.
-- 'Show orders with customer + store names.'

-- Way 1: Chained INNER JOINs (most common)
SELECT o.order_id, c.first_name, s.store_name
FROM sales.orders o
JOIN customers.customers c
    ON o.cust_id = c.customer_id
JOIN stores.stores s
    ON o.store_id = s.store_id
LIMIT 10;

-- Way 2: Subquery-driven (pre-filter before JOIN)
SELECT o.order_id, c.first_name, s.store_name
FROM (SELECT * FROM sales.orders LIMIT 100) o
JOIN customers.customers c
    ON o.cust_id = c.customer_id
JOIN stores.stores s
    ON o.store_id = s.store_id;
```

Rewriting a 3-Table JOIN — Three Ways (2/2)

```
-- Way 3: WHERE-based JOIN (older SQL-89 style)
SELECT o.order_id, c.first_name, s.store_name
FROM sales.orders o, customers.customers c,
     stores.stores s
WHERE o.cust_id = c.customer_id
     AND o.store_id = s.store_id
LIMIT 10;
-- Avoid in production – easy to forget a join cond
```

PRO TIP

PRO TIP

JOIN cheat sheet: (1) Need only matches? → INNER. (2) Need ALL from one side? → LEFT. (3) Need unmatched from BOTH sides? → FULL OUTER. (4) Need to find what's missing? → LEFT + WHERE IS NULL. When in doubt, start with LEFT JOIN — it never silently hides data.

Quick Fire Round!

Each query practices a different JOIN pattern. Modify them, swap JOIN types, check row counts. Understanding comes from experimentation.

Challenge 1: FULL OUTER — Orders vs Returns

```
-- Reconcile orders with returns
SELECT
  COALESCE(o.order_id, ret.order_id) AS order_id,
  o.net_total,
  ret.refund_amount,
  CASE
    WHEN o.order_id IS NOT NULL
      AND ret.return_id IS NOT NULL
      THEN 'Returned'
    WHEN o.order_id IS NOT NULL
      THEN 'No Return'
    ELSE 'Orphan Return'
  END AS status
FROM sales.orders o
FULL JOIN sales.returns ret
  ON o.order_id = ret.order_id
LIMIT 20;
```

Challenge 2: 4-Table Star — Order Details

```
-- Orders + customers + items + products
SELECT
  o.order_id, c.first_name,
  p.product_name, oi.quantity
FROM sales.orders o
JOIN customers.customers c
  ON o.cust_id = c.customer_id
JOIN sales.order_items oi
  ON o.order_id = oi.order_id
JOIN products.products p
  ON oi.prod_id = p.product_id
WHERE o.order_status = 'Delivered'
LIMIT 15;
```

Challenge 3: Chain — Employee → Store → Region

```
-- Following FK chain through dim_region
SELECT e.first_name, e.role,
       s.store_name, r.region_name
FROM stores.employees e
JOIN stores.stores s ON e.store_id = s.store_id
JOIN core.dim_region r
  ON s.region_id = r.region_id
ORDER BY r.region_name LIMIT 15;
```

Challenge 4: Supply Chain — Warehouse Stock

```
-- Latest inventory per warehouse
SELECT w.name, p.product_name,
       inv.quantity_on_hand
FROM supply_chain.warehouses w
JOIN supply_chain.inventory_snapshots inv
      ON w.warehouse_id = inv.warehouse_id
JOIN products.products p
      ON inv.product_id = p.product_id
WHERE inv.snapshot_date = (
      SELECT MAX(snapshot_date)
      FROM supply_chain.inventory_snapshots)
ORDER BY inv.quantity_on_hand DESC LIMIT 15;
```

Answer Before Looking!

Test your JOIN intuition. Each question targets a key concept.

Quiz 1: When to Use FULL OUTER JOIN?

EASY

When would you choose FULL OUTER JOIN over LEFT JOIN?

ANSWER: When you need unmatched rows from BOTH sides. LEFT JOIN preserves only the left-side unmatched. FULL OUTER preserves both — essential for reconciliation work where either side could have orphan rows.

Quiz 2: 5-Table JOIN Row Count

MEDIUM

You JOIN orders (1000) → customers (500) → items (5000) → products (2000) → stores (50). All INNER JOINs. Roughly how many rows will the result have?

ANSWER: Driven by the one-to-many relationships. Orders → items is one-to-many (avg 5 items per order), so 1000 orders × 5 items ≈ 5000 rows. Customers, products, stores are one-to-one dimensions — they don't multiply rows. So roughly 5000 result rows, NOT 1000.

Quiz 3: Why Did My LEFT JOIN Lose Rows?

MEDIUM

ANSWER: The LEFT JOIN silently became an INNER JOIN. WHERE o.net_total > 1000 removes all rows where net_total is NULL — including the unmatched customers. Fix: move the filter into the ON clause, or use COALESCE in WHERE.

Question



ANSWER

```
SELECT c.*, o.net_total  
FROM customers.customers c  
LEFT JOIN sales.orders o  
  ON c.customer_id = o.cust_id  
WHERE o.net_total > 1000;
```

Quiz 4: Three JOIN Rewrites

HARD

'Find orders with customer names from Mumbai.' Name three different ways to write this.

ANSWER: (1) JOIN customers and addresses, filter WHERE city = 'Mumbai'. (2) Subquery in WHERE: WHERE cust_id IN (SELECT customer_id FROM addresses WHERE city = 'Mumbai'). (3) Subquery in FROM: JOIN to a pre-filtered (SELECT * FROM customers c JOIN addresses a... WHERE a.city = 'Mumbai') derived table. All return the same result; performance varies.

Quiz 5: Supply Chain Trace Count

CRAZY

Supply chain has 3 warehouses, 5 shipments per warehouse, 100 products in inventory per warehouse. A query joins all three. If you don't filter, how many rows?

ANSWER: $3 \text{ warehouses} \times 5 \text{ shipments} \times 100 \text{ inventory} = 1,500$ rows BEFORE any filtering. But this is a Cartesian-ish situation — without proper FK matching, you can blow up to $3 \times 5 \times 100 \times \dots$ pretty quickly. ALWAYS check intermediate row counts when chaining 4+ tables.

Ambiguous Column Error

The Mistake:

In a 5-table JOIN, several tables have `store_id`. `SELECT store_id` without an alias throws: 'column reference is ambiguous'.

The Fix:

Always prefix every column with its table alias. Use short aliases consistently (o for orders, c for customers, s for stores, p for products, oi for order_items). Future you (debugging at 2 AM) will thank you.

Joining Without Tracing FKs First

The Mistake:

Picking JOIN columns based on column NAMES rather than actual foreign-key relationships. Example: assuming `sales.payments.customer_id` exists — it doesn't, `payments` links via `order_id` only.

The Fix:

Before writing the SQL, draw the FK path on paper. RetailMart cheat sheet: `orders.cust_id` → `customers.customer_id`; `payments.order_id` → `orders.order_id`; `order_items.prod_id` → `products.product_id`; `products.brand_id` → `dim_brand` → `dim_category`.

Fan-Out in Multi-Table Aggregates

The Mistake:

JOIN employees AND orders directly to stores. If a store has 10 employees and 500 orders, you get 5,000 rows per store — every SUM is 10× inflated.

The Fix:

Pre-aggregate each one-to-many side in a subquery before joining. We cover this pattern formally on Subqueries Part 2. For now: `COUNT(DISTINCT employee_id)` and `COUNT(DISTINCT order_id)` instead of `COUNT(*)`.

FULL OUTER JOIN with WHERE on One Side

The Mistake:

FULL JOIN ... WHERE o.status = 'X' removes right-only rows where status is NULL. The FULL becomes a partial join silently.

The Fix:

Side-specific filters belong in the ON clause OR wrap with COALESCE in WHERE: WHERE COALESCE(o.status, 'Unknown') = 'X'. Or pre-filter each source in a subquery before the FULL OUTER JOIN.

When Would You Use FULL OUTER JOIN?

Q: 'Give me a real use case for FULL OUTER JOIN.'

A: Data reconciliation. Comparing two systems' records to find: (1) records in System A missing from B, (2) records in B missing from A, (3) matched records. Concrete examples: bank statement vs internal ledger, GST returns vs accounting books, source system vs target system during a data migration.

Star Pattern vs Chain Pattern

Q: 'Explain star and chain patterns in multi-table JOINS.'

A: Star = ONE central fact table (orders) JOINed to multiple dimension tables (customers, stores, products). Chain = following FK relationships sequentially (employee → store → region). Real queries usually MIX both. The pattern names come from data warehousing — star schemas vs snowflake schemas.

Why Did My JOIN Return Too Many Rows?

Q: 'My 5-table JOIN returns 100x more rows than expected. What went wrong?'

A: Fan-out (row multiplication). When you JOIN through a one-to-many relationship, each parent row multiplies by the number of children. Multiple one-to-many JOINS compound. Quick diagnostic: compare `COUNT(*)` to `COUNT(DISTINCT primary_key)` — if they differ, you have fan-out.

INNER vs LEFT JOIN Performance

Q: 'Is INNER JOIN always faster than LEFT JOIN?'

A: Often, but not always. INNER JOIN lets the optimizer drop unmatched rows early. LEFT must preserve them.

Difference is usually small. ALWAYS choose based on CORRECTNESS first — do you need unmatched rows? If yes, LEFT JOIN. If no, INNER JOIN.

Rewriting a JOIN — Three Forms

Q: 'Show me three ways to write the same JOIN.'

A: (1) Standard JOIN syntax: `FROM a JOIN b ON a.x = b.x`. (2) Subquery in WHERE: `WHERE a.x IN (SELECT x FROM b)`. (3) Subquery in FROM: `JOIN (SELECT ... FROM b) sub ON ...`. Same result, different shapes. The optimizer often rewrites them to the same execution plan.

FULL OUTER JOIN Result Bounds

Q: 'How many rows does a FULL OUTER JOIN return?'

A: At least $\text{MAX}(\text{rows_in_A}, \text{rows_in_B})$. At most: $\text{A_unmatched} + \text{B_unmatched} + \text{matched_combinations}$. So if A has 100 rows and B has 150, the minimum is 150 rows (every B row appears at least once).

Real Interview: Order 360° View

Q: 'Write a query showing every detail about a customer order — customer info, products, payment, shipment, store, warehouse.'

A: Anchor on orders. INNER JOIN customers, stores, order_items, products (required). LEFT JOIN payments, shipments, warehouse (optional). 6–7 tables, mixed JOIN types. Explain why each is INNER or LEFT — that's what interviewers actually evaluate.

KEY TAKEAWAYS (1/2)

FULL OUTER JOIN:

1. Returns ALL rows from BOTH tables — nothing excluded.
2. Use for data reconciliation, audits, finding gaps.
3. CASE classifies: Matched / Left-Only / Right-Only.

MULTI-TABLE JOINS:

4. Star pattern: one fact + many dimensions. Chain pattern: $A \rightarrow B \rightarrow C$.
5. Mix INNER (required) and LEFT (optional) within one query.
6. Trace FK path on paper FIRST — before writing SQL.

KEY TAKEAWAYS (2/2)

7. One-to-many JOINS multiply rows — fan-out is the #1 bug.

JOIN STRATEGY:

8. Default to LEFT JOIN when unsure — never silently hides data.

9. The same business question has multiple valid JOIN shapes.

10. Use EXPLAIN to verify your JOIN actually does what you think.

What You Achieved Today

You learned the THREE skills that separate junior from senior analysts: (1) FULL OUTER JOIN for reconciliation, (2) multi-table JOINS spanning 5+ tables, (3) JOIN strategy — picking the right shape for the question.

Every analyst query that powers a real dashboard uses these patterns. Zomato's order tracking, Amazon's product page, Paytm's transaction history — multi-table JOINS all the way down.

Tomorrow: SELF JOIN (joining a table to itself for hierarchies and pairs) + UNION (stacking result sets vertically).

FULL OUTER & Multi-Table JOINS

-- FULL OUTER JOIN

```
FROM a FULL JOIN b ON a.key = b.key
-- ALL rows from BOTH tables
-- NULLs on the unmatched side
```

-- FULL OUTER – Match Status

```
CASE WHEN a.key IS NOT NULL
      AND b.key IS NOT NULL THEN 'Match'
      WHEN a.key IS NOT NULL THEN 'Left Only'
      ELSE 'Right Only' END
```

-- Star Pattern

```
FROM fact JOIN dim1 ON ...
      JOIN dim2 ON ...
      LEFT JOIN optional ON ...
```

FULL OUTER & Multi-Table JOINS

-- Chain Pattern

```
FROM a JOIN b ON a.x = b.x
      JOIN c ON b.y = c.y
-- Follow FK path A → B → C
```

-- Anti-Join (Find Gaps)

```
FROM a LEFT JOIN b ON a.key = b.key
WHERE b.key IS NULL
-- Rows in A with no match in B
```

-- Mix JOIN Types

```
JOIN required_table ON ... -- INNER
LEFT JOIN optional ON ... -- preserve
-- Pick per relationship
```

Joins Part 2 Take-Home Challenge (1/3)

Build these queries on RetailMart V3:

Easy (FULL OUTER + 4-Table):

- 1.: FULL JOIN departments vs employees. Classify: Matched / Empty Dept / Invalid Dept.
- 2.: 4-table order receipt: customer name, store, product, quantity. Delivered only.

Medium:

Joins Part 2 Take-Home Challenge (2/3)

3.: Unsold inventory: products in catalog with NO orders. JOIN to brand.

4.: Supply chain trace: product → shipment → warehouse → inventory.

Hard:

5.: 360° order view: 6-table JOIN with INNER + LEFT mix. Customer + store + items + products + payment + shipment.

6.:

Joins Part 2 Take-Home Challenge (3/3)

Rewrite ONE 3-table JOIN three different ways: standard JOIN, subquery in WHERE, subquery in FROM.

Crazy:

7.: Migration reconciliation: pre-aggregate tickets per customer, then FULL OUTER JOIN customers. Classify Matched / No Tickets / Orphan.

Push your SQL file to GitHub!

<https://github.com/starlordali4444/PostgreSql>

Coming Tomorrow

SELF JOIN — joining a table to itself. Find employee-manager hierarchies. Find product pairs in the same category. Same table, two aliases, row-by-row comparison.

UNION — stacking results vertically. Different from JOINS (which stitch columns) — UNION stacks ROWS. Combine query results from disparate sources into one unified output. Subqueries Part 1 completes the JOIN toolkit!